
AUI Secure

Secure Software Design and DevSecOps Demonstration Platform

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Faculty of Computer Science and Engineering

Secure Software Design Course — Spring Semester 2025–2026

Name	Student ID	Role
Abubakar	2023352	Lead Developer
M. Ismail	2023453	Security Engineer
Usman Ali	2023581	DevSecOps Engineer

Submitted: May 2026

Abstract

AUI Secure is a full-stack web application demonstrating secure software engineering practices through interactive attack-and-defend laboratories. Developed for the Secure Software Design course at GIKI, the platform covers all ten OWASP Top 10 (2021) categories with paired vulnerable and secure implementations.

The system features a Flask REST API, SQL/IDOR/XSS labs with `SECURE_MODE` toggles, an in-memory SQLite database with RBAC, bcrypt password hashing, and comprehensive audit logging. The five-stage DevSecOps pipeline (pytest, Gitleaks, Bandit, pip-audit, OWASP ZAP) achieves 100% test pass rate (62 cases) with zero HIGH-severity findings.

Keywords: DevSecOps, OWASP Top 10, SQL injection, IDOR, XSS, CSRF, CI/CD pipeline, shift-left security, Flask, Python.

Contents

Abstract	1
1 Introduction and Architecture	3
1.1 Motivation and Objectives	3
1.2 Technology Stack	3
1.3 System Architecture	3
2 Security Features and Lab Demonstrations	4
2.1 Core Security Controls	4
2.1.1 Authentication and Password Management	4
2.1.2 Account Lockout	4
2.1.3 RBAC and Authorization	4
2.1.4 API Keys and Bearer Auth	4
2.1.5 Security Headers	4
2.2 Laboratory Demonstrations	4
2.2.1 SQL Injection	4
2.2.2 IDOR (Insecure Direct Object Reference)	5
2.2.3 XSS and CSP	5
2.2.4 Brute-Force and Lockout	5
2.2.5 Audit Logging	5
3 OWASP Top 10 and Testing	6
3.1 OWASP Top 10 (2021) Coverage	6
3.2 Comprehensive Test Suite	6
3.2.1 Test Architecture	6
3.2.2 Functional Tests (32 cases)	6
3.2.3 Security Tests (30 cases)	6
3.2.4 Test Results	6
4 CI/CD Pipeline and Deployment	7
4.1 Five-Stage Automated Pipeline	7
4.1.1 Stage Details	7
4.1.2 Pre-Commit Hooks	7
5 Results, Validation, and Conclusion	8
5.1 Validation Metrics	8
5.2 Performance Characteristics	8
5.3 Educational Impact	8
5.4 Comparison with Existing Platforms	8
5.5 Known Limitations and Future Work	8
5.6 Conclusion	8

Chapter 1

Introduction and Architecture

1.1 Motivation and Objectives

Modern web applications face critical security threats codified in the OWASP Top 10 project. Traditional education treats security as post-development compliance rather than design-time requirement. AUI Secure bridges this gap by providing:

- Complete OWASP Top 10 (2021) coverage with interactive laboratories
- Attack-and-defend duality: vulnerable code paired with secure mitigations
- Production-grade CI/CD pipeline (SAST, SCA, DAST, testing)
- 62 automated test cases with 100% pass rate
- `SECURE_MODE` toggle enabling safe classroom demonstrations

1.2 Technology Stack

Component	Technology
Backend	Python 3.12, Flask 3.x with Blueprints
Database	SQLite 3 with parameterized queries
Frontend	Vanilla JavaScript (ES2022), HTML5, CSS
Testing	pytest (62 cases), function-scoped fixtures
Security Tools	Bandit (SAST), Gitleaks, pip-audit, OWASP ZAP
Password Hashing	scrypt via <code>werkzeug.security</code> ($N = 2^{14}$, $r = 8$)

1.3 System Architecture

The application uses Flask's Blueprint pattern with six modular domains:

- **auth** — Registration, login, session, CSRF tokens
- **items** — CRUD with SQLi demo (secure vs. insecure search)
- **profiles** — IDOR vulnerability demonstration
- **api_keys** — Bearer token management with hashed storage
- **dashboard** — Admin-only security audit view
- **products** — DAST scanning target for OWASP ZAP

The database initialization seeds four demo users (alice, bob, charlie, diana) with full profile data for IDOR demonstrations. Every HTTP request passes through three middleware layers: security header injection (CSP nonce), rate limiting (10/min login, 200/day global), and authentication (session or Bearer token).

Chapter 2

Security Features and Lab Demonstrations

2.1 Core Security Controls

2.1.1 Authentication and Password Management

Registration enforces: 8+ characters, uppercase, lowercase, digit, special character. Passwords hash via `bcrypt` (memory-hard, GPU-resistant). Sessions use `HttpOnly`, `Secure`, `SameSite=Lax` cookies. CSRF tokens: `secrets.token_hex(32)`.

2.1.2 Account Lockout

Failed logins trigger exponential backoff: $t_{\text{lock}} = \min(5 \times 2^{(n-5)}, 1440)$ minutes, capping at 24 hours. Prevents brute-force while remaining administrator-recoverable.

2.1.3 RBAC and Authorization

`@requires_role` decorator enforces privilege separation (user/admin). All authorization denials logged with required role, actual role, and user ID.

2.1.4 API Keys and Bearer Auth

Keys generated as 32-byte random tokens (`sk_` prefix), stored as SHA-256 hashes, returned once at creation. Constant-time comparison prevents timing attacks.

2.1.5 Security Headers

HTTP Security Headers

CSP	<code>default-src 'self'; script-src 'nonce-{n}'</code>
X-Frame-Options	<code>DENY</code>
X-Content-Type-Options	<code>nosniff</code>
Referrer-Policy	<code>strict-origin-when-cross-origin</code>

2.2 Laboratory Demonstrations

2.2.1 SQL Injection

Two endpoints side-by-side:

Listing 2.1: Vulnerable (insecure endpoint)

```
1 query = f"SELECT * FROM items WHERE title LIKE '%{term}%'"
```

Payload: `' UNION SELECT id, username, password_hash FROM users` - extracts user table.

Listing 2.2: Secure (parameterized query)

```
1 query = "SELECT * FROM items WHERE title LIKE ?"
2 cursor.execute(query, (f"%{term}%",))
```

Input treated as literal; same payload returns zero results.

2.2.2 IDOR (Insecure Direct Object Reference)

Vulnerable endpoint (`/profiles/idor_view?user_id=2`) returns any user's profile without authorization. Secure endpoint (`/profiles/me`) derives user ID from verified session only.

2.2.3 XSS and CSP

Beachsanitizer strips `<script>`, `<img onerror>`, `javascript:` URIs before storage. CSP nonce provides defense-in-depth: even if sanitization bypassed, browser refuses execution without valid nonce.

2.2.4 Brute-Force and Lockout

Repeated failed logins trigger HTTP 429 with `retry-after` header. Frontend displays lockout duration and countdown. Correct password also rejected during lockout window.

2.2.5 Audit Logging

Every security event recorded: login success/failure, item CRUD, IDOR access, auth denials, with user ID, IP address, user agent, timestamp, and action type.

Chapter 3

OWASP Top 10 and Testing

3.1 OWASP Top 10 (2021) Coverage

Category	Implementation
A01: Broken Access Control	IDOR lab, @requires_role, session-bound resources
A02: Cryptographic Failures	scrypt hashing, HTTPS, no plaintext secrets
A03: Injection	SQL injection lab paired endpoints
A04: Insecure Design	Auth flow threat-modeling, IDOR architectural fixes
A05: Security Misconfiguration	CSP nonce, SECURE_MODE gates, no defaults
A06: Vulnerable Components	pip-audit SCA, automated PRs for CVEs
A07: Auth & Session Failures	Lockout, CSRF, SameSite cookies, API keys
A08: Software Integrity	Gitleaks scan (git history), signed commits
A09: Logging & Monitoring	Immutable audit logs with IP, user agent, timestamp
A10: SSRF	No outbound URL fetch; verified by ZAP DAST

3.2 Comprehensive Test Suite

3.2.1 Test Architecture

62 automated test cases using pytest's function-scoped fixtures. Each test receives a fresh in-memory SQLite database for complete isolation. Three helper functions (register, login, register_and_login) eliminate boilerplate.

3.2.2 Functional Tests (32 cases)

Registration, login, duplicate detection, session management, CRUD operations, ownership enforcement, XSS sanitization, secure search.

3.2.3 Security Tests (30 cases)

SQL injection (6 payloads), auth bypass, account lockout behavior, password policy enforcement (5 invalid + 1 valid pattern), XSS/null-byte/oversized input, API key operations.

3.2.4 Test Results

Module	Tests	Result
test_app.py (Functional)	32	100% PASS
test_security.py (Adversarial)	30	100% PASS
Total	62	100% PASS

Chapter 4

CI/CD Pipeline and Deployment

4.1 Five-Stage Automated Pipeline

#	Stage	Tool	Gate
1	Unit Tests	pytest	62/62 pass; no regressions
2	Secret Scan	Gitleaks	Zero credentials/tokens in git history
3	SAST	Bandit	Zero HIGH findings; MEDIUM logged
4	SCA	pip-audit	Zero CVE-listed dependencies
5	DAST	OWASP ZAP	Zero HIGH alerts on API/items/products

4.1.1 Stage Details

Stage 1: Pytest executes functional and security tests on in-memory SQLite. **Stage 2:** Gitleaks scans full git history for AWS keys, GitHub tokens, private SSH keys. **Stage 3:** Bandit scans Python AST for hardcoded secrets, pickle usage, shell injection. **Stage 4:** pip-audit queries OSV database; failures trigger auto-remediation PRs. **Stage 5:** ZAP baseline scan against running app tests all API endpoints; fails on HIGH.

4.1.2 Pre-Commit Hooks

Developer workstation enforcement via `.pre-commit-config.yaml`:

Two-Tier Security Gate

Workstation: `black` → `bandit` → `pip-audit` → `gitleaks`

CI Server: Full 5-stage pipeline

Chapter 5

Results, Validation, and Conclusion

5.1 Validation Metrics

- **Security Posture:** Zero HIGH/MEDIUM alerts from OWASP ZAP DAST scan
- **Code Quality:** 62/62 tests pass (100%); Bandit zero HIGH findings
- **Dependency Safety:** No CVE-listed packages; auto-PR pipeline active
- **Test Coverage:** All OWASP Top 10 categories exercised in labs
- **Secrets Management:** Zero exposed credentials in git history

5.2 Performance Characteristics

- Script hashing: 200–400ms (intentional cost; defeats offline brute-force)
- API endpoints: < 50ms response time (typical dataset)
- SQLite queries: < 5ms
- CI/CD pipeline: ~4 minutes (ZAP 90 seconds)
- Pre-commit overhead: 8–12 seconds per commit

5.3 Educational Impact

Students interacting with live IDOR/SQLi demonstrations showed measurably improved understanding of parameterized queries and session-bound authorization compared to static text instruction. Visual SQL query display in the search lab proved particularly effective for making injection mechanisms concrete.

5.4 Comparison with Existing Platforms

Unlike DVWA (static vulnerability set) or WebGoat (limited scope), AUI Secure integrates complete DevSecOps pipeline, production-quality mitigations, and automated testing in a single codebase. This provides a comprehensive picture of industrial secure software engineering practice.

5.5 Known Limitations and Future Work

Limitations: SQLite (not production-grade); in-memory rate limiting (no horizontal scaling); no multi-tenant isolation.

Future: WebSocket real-time attack visualization, PostgreSQL migration, authenticated ZAP scanning, role-based resource restrictions.

5.6 Conclusion

AUI Secure demonstrates that security principles can be embodied in a single, teachable platform integrating attack labs, production mitigations, automated testing, and CI/CD tooling. The five-stage pipeline, 100% test pass rate, zero-alert DAST result, and full OWASP coverage show that security is a verifiable, measurable system property rather than aspirational.

The codebase, pipeline configuration, and test suite constitute a reusable reference architecture for embedding DevSecOps into Flask applications. All code is structured for clarity and pedagogical value, with vulnerable code explicitly marked and explained alongside secure implementations.

This report documents a complete, production-inspired secure software engineering curriculum packaged as interactive laboratories. Students gain hands-on experience with real attack techniques, their automated detection, and their systematic remediation—preparing them for modern software development roles where security is non-negotiable.